```
const sessionListEl = document.getElementById('sessionList');
const messagesEl = document.getElementById('messages');
const emptyStateEl = document.getElementById('emptyState');
const chatTitleEl = document.getElementById('chatTitle');
const composerEl = document.getElementById('composer');
const inputEl = document.getElementById('messageInput');
const sendBtn = document.getElementById('sendBtn');
const newChatBtn = document.getElementById('newChatBtn');

let sessions = [];
let currentSessionId = localStorage.getItem('ember:lastSession') || null;

// ---------- Utilities ----------
function escapeHtml(str) {
  return str
    .replace(/&/g, '&amp;')
    .replace(/</g, '&lt;')
    .replace(/>/g, '&gt;');
}

// Very small, safe markdown-ish renderer: code blocks, inline code, bold, line b
function renderContent(text) {
  let safe = escapeHtml(text);
  safe = safe.replace(/```([\s\S]*?)```/g, (_, code) => `<pre><code>${code.trim()
  safe = safe.replace(/`([^`]+)`/g, '<code>$1</code>');
  safe = safe.replace(/\*\*([^*]+)\*\*/g, '<strong>$1</strong>');
  return safe;
}

function autoResize() {
  inputEl.style.height = 'auto';
  inputEl.style.height = Math.min(inputEl.scrollHeight, 160) + 'px';
}

// ---------- Rendering ----------
function renderSessionList() {
  sessionListEl.innerHTML = '';
  sessions.forEach((s) => {
    const item = document.createElement('div');
    item.className = 'session-item' + (s.id === currentSessionId ? ' active' : ''
    item.innerHTML = `<span class="title">${escapeHtml(s.title)}</span><button cl
    item.querySelector('.title').addEventListener('click', () => switchSession(s.
    item.querySelector('.delete-btn').addEventListener('click', (e) => {
      e.stopPropagation();
```

```
      deleteSession(s.id);
    });
    sessionListEl.appendChild(item);
  });
}

function appendMessage(role, content) {
  emptyStateEl.style.display = 'none';
  const msg = document.createElement('div');
  msg.className = `msg ${role}`;
  msg.innerHTML = `
    <div class="avatar">${role === 'user' ? 'You' : '🔥'}</div>
    <div class="bubble">${renderContent(content)}</div>
  `;
  messagesEl.appendChild(msg);
  messagesEl.scrollTop = messagesEl.scrollHeight;
  return msg;
}

function appendTyping() {
  const msg = document.createElement('div');
  msg.className = 'msg assistant';
  msg.id = 'typingIndicator';
  msg.innerHTML = `
    <div class="avatar">🔥</div>
    <div class="bubble"><div class="typing"><span></span><span></span><span></spa
  `;
  messagesEl.appendChild(msg);
  messagesEl.scrollTop = messagesEl.scrollHeight;
}

function removeTyping() {
  document.getElementById('typingIndicator')?.remove();
}

// ---------- API calls ----------
async function fetchSessions() {
  const res = await fetch('/api/sessions');
  sessions = await res.json();
  renderSessionList();
}

async function createSession() {
  const res = await fetch('/api/sessions', { method: 'POST' });
  const session = await res.json();
  sessions.unshift(session);
  currentSessionId = session.id;
```

```
  localStorage.setItem('ember:lastSession', currentSessionId);
  renderSessionList();
  chatTitleEl.textContent = session.title;
  messagesEl.innerHTML = '';
  emptyStateEl.style.display = 'block';
  messagesEl.appendChild(emptyStateEl);
}

async function switchSession(id) {
  currentSessionId = id;
  localStorage.setItem('ember:lastSession', id);
  renderSessionList();
  const session = sessions.find((s) => s.id === id);
  chatTitleEl.textContent = session ? session.title : 'Chat';

  messagesEl.innerHTML = '';
  const res = await fetch(`/api/sessions/${id}/messages`);
  const msgs = await res.json();

  if (msgs.length === 0) {
    messagesEl.appendChild(emptyStateEl);
    emptyStateEl.style.display = 'block';
  } else {
    msgs.forEach((m) => appendMessage(m.role, m.content));
  }
}

async function deleteSession(id) {
  await fetch(`/api/sessions/${id}`, { method: 'DELETE' });
  sessions = sessions.filter((s) => s.id !== id);
  if (currentSessionId === id) {
    currentSessionId = null;
    localStorage.removeItem('ember:lastSession');
    if (sessions.length > 0) {
      switchSession(sessions[0].id);
    } else {
      await createSession();
    }
  }
  renderSessionList();
}

async function sendMessage(text) {
  if (!currentSessionId) await createSession();

  appendMessage('user', text);
  appendTyping();
```

```
  sendBtn.disabled = true;

  try {
    const res = await fetch(`/api/sessions/${currentSessionId}/messages`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ message: text }),
    });
    const data = await res.json();
    removeTyping();

    if (!res.ok) {
      appendMessage('assistant', `⚠️ ${data.error || 'Something went wrong.'}`);
    } else {
      appendMessage('assistant', data.reply);
      // Refresh session titles (first message auto-renames the chat)
      await fetchSessions();
    }
  } catch (err) {
    removeTyping();
    appendMessage('assistant', `⚠️ Network error: ${err.message}`);
  } finally {
    sendBtn.disabled = false;
  }
}

// ---------- Events ----------
composerEl.addEventListener('submit', (e) => {
  e.preventDefault();
  const text = inputEl.value.trim();
  if (!text) return;
  inputEl.value = '';
  autoResize();
  sendMessage(text);
});

inputEl.addEventListener('keydown', (e) => {
  if (e.key === 'Enter' && !e.shiftKey) {
    e.preventDefault();
    composerEl.requestSubmit();
  }
});

inputEl.addEventListener('input', autoResize);
newChatBtn.addEventListener('click', createSession);

// ---------- Init ----------
```

```
(async function init() {
  await fetchSessions();

  if (currentSessionId && sessions.some((s) => s.id === currentSessionId)) {
    await switchSession(currentSessionId);
  } else if (sessions.length > 0) {
    await switchSession(sessions[0].id);
  } else {
    await createSession();
  }
})();
```